Interoperation Parallelism

1 Overview

Interoperation parallelism refers to executing different operations in a query expression in parallel. Unlike intraoperation parallelism (which splits a single task), interoperation parallelism focuses on the orchestration of multiple distinct tasks.

Forms of Interoperation Parallelism

There are two primary forms:

1. **Pipelined Parallelism:** Operations are connected in a chain where one consumes tuples as they are produced by another.
2. **Independent Parallelism:** Operations that do not depend on each other are executed simultaneously.

2 Pipelined Parallelism

In pipelining, the output tuples of one operation (A) are consumed by a second operation (B) before A has produced its entire output. Parallel systems use this to run A and B simultaneously on different processors.

4-Way Join Pipeline

Consider the join: $r_1 \bowtie r_2 \bowtie r_3 \bowtie r_4$. We can assign processors as follows:

- **Processor P_1:** Computes $temp_1 = r_1 \bowtie r_2$.
- **Processor P_2:** Consumes tuples from $temp_1$ to compute $temp_2 = temp_1 \bowtie r_3$.
- **Processor P_3:** Consumes tuples from $temp_2$ to compute $temp_2 \bowtie r_4$.

P_2 can begin its join as soon as P_1 produces its *first* matching tuple. This avoids writing large intermediate results ($temp_1, temp_2$) to disk.

Factors Limiting Pipeline Utility

Pipelined parallelism is useful for reducing I/O but doesn't scale well for several reasons:

1. **Chain Length:** Pipelines are usually too short to provide massive parallelism.
2. **Blocking Operators:** Operators like **Sort** or **Aggregate** cannot produce any output until all input is read, breaking the pipeline.
3. **Execution Skew:** If one operator is much slower than the rest, the others will spend most of their time waiting (idle).

3 Independent Parallelism

Operations in a query that do not have data dependencies can be executed independently.

Independent Join Scenario

For the same 4-way join: $(r_1 \bowtie r_2) \bowtie (r_3 \bowtie r_4)$:

1. P_1 computes $temp_1 = r_1 \bowtie r_2$.
2. P_2 computes $temp_2 = r_3 \bowtie r_4$.
3. P_1 and P_2 work **simultaneously** and independently.
4. Once both finish, a third processor P_3 computes $temp_1 \bowtie temp_2$.

Combining Techniques

We can combine both forms: P_1 and P_2 run independently, but as they produce tuples, they **pipeline** results as input to P_3. This maximizes processor utilization.

Scalability Note

Like pipelined parallelism, independent parallelism does not provide a high degree of parallelism. In systems with hundreds of processors, **partitioned (intraoperation) parallelism** remains the dominant source of performance gains.

While interoperation parallelism handles query-level orchestration, it is most effective when paired with intraoperation partitioning.